RCA / Postmortem — Incident #3: High CPU Exhaustion

Field	Value
Incident ID	INC-003
Title	Notes API replicas driven to sustained high CPU by unthrottled compute-bound endpoint
Severity	SEV-3 (elevated resource usage; no outage; pods stayed within limits)
Status	Resolved (auto-recovered)
Author	Vishnu K
Date	2026-08-02
Simulated?	Yes — controlled failure injection for resilience testing
Primary alert	`NotesApiHighCPU`

1. Summary

Sustained CPU-bound load was generated against the application's `/api/stress` endpoint, which runs a deliberate busy-loop. Twelve rounds of parallel requests (each burning CPU for ~20s) were distributed evenly across both replicas by the load balancer, driving each pod to **~0.50–0.55 CPU cores** — well above the `NotesApiHighCPU` alert threshold of 0.4 cores, but comfortably below each pod's 1-core hard limit.

No pod was restarted or evicted; the resource limits acted as a safety ceiling. The `NotesApiHighCPU` alert fired for **both** pods. Recovery was fully automatic: once the load stopped, CPU returned to baseline within ~30 seconds and the alert auto-resolved after the metric averaging window aged out the spike.

2. Impact

- **User-facing:** Minimal. Normal requests would experience elevated latency during the CPU saturation, but the app remained available (pods stayed `1/1 Running`).
- **Resource:** Both replicas ran at ~50% of their CPU limit for ~3 minutes.
- **Data:** No data loss.
- **Duration:** ~6 min 8 sec total (including alert auto-resolution lag).

3. Timeline (UTC)

Time	Event
23:12:52	**T+0** — Load injection begins (parallel `/api/stress` requests)
23:13:22	Pod CPU crosses the 0.4-core threshold

Time	Event
23:13:30	`NotesApiHighCPU` enters **Pending**
23:15:30	Alert transitions to **Firing** — **time-to-detection: ~2m 38s**
23:15:58	Load script auto-completes (~3m 6s of load)
23:16:30	CPU returns to baseline (~30s after load stops)
23:19:00	Alert auto-resolves (`rate([3m])` window ages out spike samples)

---

4. Detection

- **Primary alert:** `NotesApiHighCPU` — fires when a pod's CPU usage (`rate(container_cpu_usage_seconds_total[3m])`) exceeds 0.4 cores, sustained for 2 minutes. Values at firing: **0.550 and 0.548 cores** per pod — **both** replicas breached, so the alert fired for both.
- **Supporting signals:** Grafana CPU panels showed the ramp; Loki captured `Stress test completed after 20s (iterations: 32,808,039)` — direct proof the busy-loop ran to exhaustion.
- **Time-to-detection:** ~2 min 38 sec.

Auto-resolution lag (worth understanding)

CPU dropped to baseline within ~30 seconds of the load stopping, but the alert took an **additional ~3 minutes** to auto-resolve. This is expected behaviour: the alert expression averages over a `rate([3m])` window, so the elevated samples must "age out" of that trailing window before the computed average falls back below the 0.4-core threshold. This is a normal property of window-based metrics — recovery of the *signal* lags recovery of the *system*. Understanding this prevents an operator from wrongly concluding "the fix didn't work" during the lag.

---

5. Root Cause Analysis (5 Whys)

- **Why did the pods show high CPU?** → They executed a CPU-intensive busy-loop repeatedly.
- **Why were they running a busy-loop?** → The `/api/stress` endpoint deliberately burns CPU, and it was called continuously in parallel.
- **Why did continuous calls saturate both pods?** → The load balancer round-robined the requests evenly across both replicas.
- **Why was there no protection against this?** → The endpoint has no rate-limiting, no concurrency cap, and no per-request CPU accounting — any client can request unbounded CPU work.
- **Why does that matter beyond this test?** → Any unthrottled compute-bound endpoint is a denial-of-service vector: a client (or a bug, or an attacker) can exhaust CPU and degrade the service for everyone. ← **Root cause: no rate-limiting / resource-guarding on a compute-bound endpoint.**

Mitigating factor (by design): the Kubernetes CPU *limit* of 1 core per pod acted as a blast-radius ceiling — the busy-loop could not consume the whole node, only the pod's allotted share. This is exactly why resource limits are set.

---

6. What Went Well

- **Resource limits contained the blast radius.** Each pod was capped at 1 core, so the stress could not starve the node or other workloads. The limits set in the Helm chart did their job.
- **No restarts, no evictions.** The pods absorbed the load and stayed healthy — CPU pressure throttles, it doesn't crash (unlike memory pressure, which OOM-kills).
- **Even load distribution.** The load balancer spread requests across both replicas, confirming HA/load-balancing works under load, not just at idle.
- **Fully automatic recovery** — no operator intervention required.

---

7. What Could Be Improved

- **No rate-limiting** on `/api/stress` (or any endpoint) — it's an open CPU-exhaustion vector.
- **Alert auto-resolution lag (~3 min)** is inherent to the metric window; if faster resolution signalling is desired, a shorter `rate()` window trades smoothness for responsiveness.
- **No Horizontal Pod Autoscaler (HPA)** — under sustained legitimate CPU load, a production service would scale out; here the fixed 2 replicas simply saturated.

---

8. Corrective Actions

#	Action	Owner	Priority	Target Date
1	Document the high-CPU / scaling response in the runbook (scale out, identify hot endpoint)	Vishnu K	High	2026-08-04
2	Add rate-limiting / concurrency caps to compute-bound endpoints (or gate `/api/stress` behind auth / disable in prod)	Vishnu K	Medium	Backlog
3	Add a Horizontal Pod Autoscaler (HPA) on CPU so the service scales out under sustained load	Vishnu K	Medium	Backlog
4	Consider a shorter alert window or a burn-rate alert for faster CPU-spike detection where warranted	Vishnu K	Low	Backlog

---

9. Evidence

Baseline & injection

```
Last login: Sun Aug  2 22:40:50 2026 from 157.51.219.190
ubuntu@ip-172-31-33-133:~$ sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -n default
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-545877b456-7965p          1/1     Running   0          4h20m
notes-api-545877b456-hqjzh          1/1     Running   0          4h20m
notes-api-postgres-5dbcdc9df5-4qsql   1/1     Running   0          26m
NAME                                CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p          3m           69Mi
notes-api-545877b456-hqjzh          2m           69Mi
notes-api-postgres-5dbcdc9df5-4qsql   8m           19Mi
ubuntu@ip-172-31-33-133:~$
```

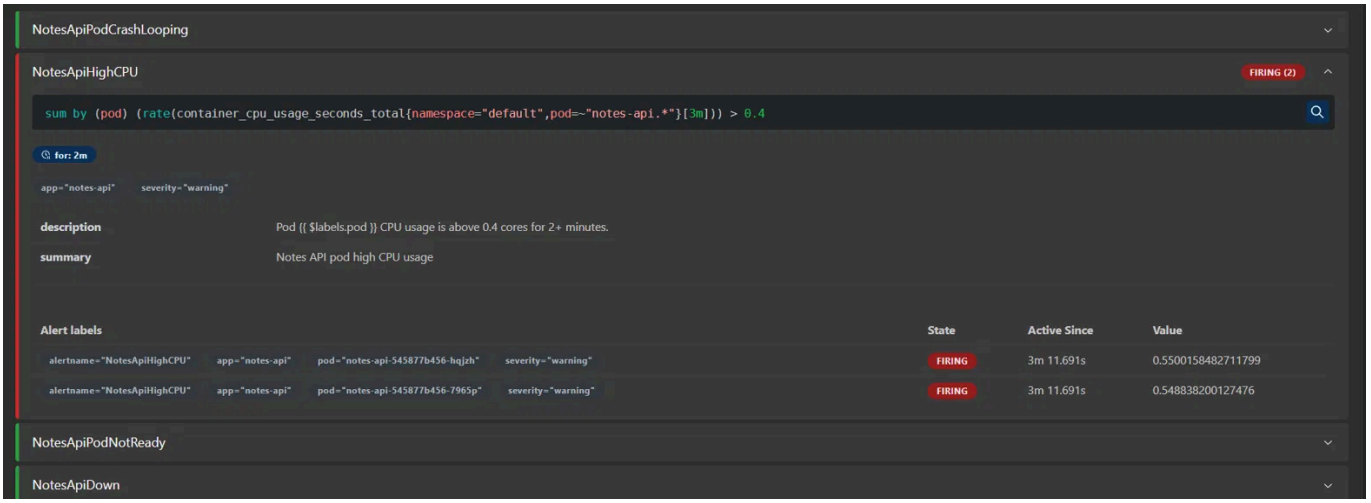
```
vishn@VISHNU MINGW64 ~/Downloads
$ ssh -i "sre-test-key.pem" ubuntu@ec2-13-126-63-217.ap-south-1.compute.amazonaws.com
Last login: Sun Aug  2 22:40:50 2026 from 157.51.219.190
ubuntu@ip-172-31-33-133:~$ sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -n default
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-545877b456-7965p          1/1     Running   0          4h20m
notes-api-545877b456-hqjzh          1/1     Running   0          4h20m
notes-api-postgres-5dbcdc9df5-4qsql   1/1     Running   0          26m
NAME                                CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p          3m           69Mi
notes-api-545877b456-hqjzh          2m           69Mi
notes-api-postgres-5dbcdc9df5-4qsql   8m           19Mi
ubuntu@ip-172-31-33-133:~$ echo "INCIDENT 3 START (T+0): $(date -u '+%Y-%m-%d %H:%M:%S UTC')"
cd ~/SRE-Project1
./tests/simulate-high-cpu.sh
INCIDENT 3 START (T+0): 2026-08-02 23:12:52 UTC
========================================
 FAILURE SIMULATION: High CPU Exhaustion
 Start time: 2026-08-02 23:12:52 UTC
========================================
[STEP 1] Baseline — current pod CPU (via kubectl top):
NAME                        CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p   2m           69Mi
notes-api-545877b456-hqjzh   3m           69Mi

[STEP 2] Generating sustained CPU load via /api/stress...
         Sending 12 rounds of stress (each burns CPU for ~20s).
         Running them in parallel to hit both replicas.

  Round 1: firing parallel stress requests (23:12:52)
  Round 2: firing parallel stress requests (23:13:12)
  Round 3: firing parallel stress requests (23:13:32)
  Round 4: firing parallel stress requests (23:13:52)
  Round 5: firing parallel stress requests (23:14:12)
  Round 6: firing parallel stress requests (23:14:32)
```

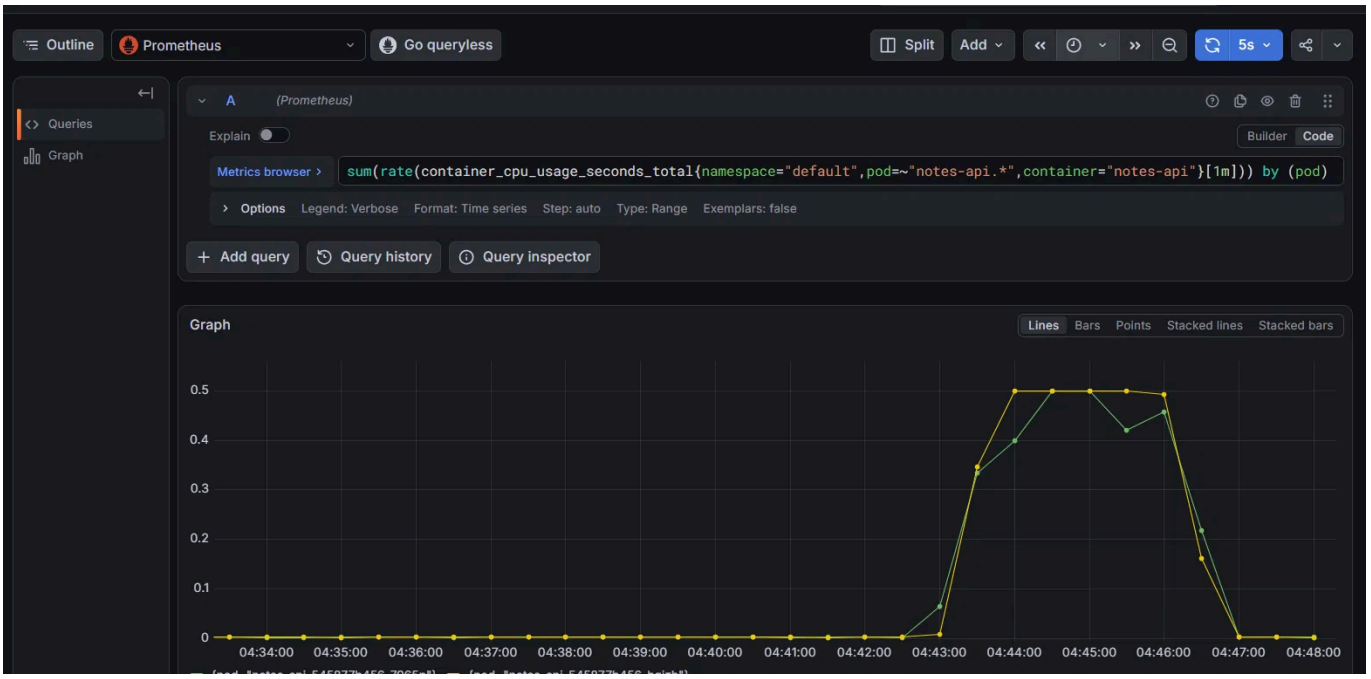
```
ubuntu@ip-172-31-33-133:~$ sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -n default
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-545877b456-7965p          1/1     Running   0          4h20m
notes-api-545877b456-hqjzh          1/1     Running   0          4h20m
notes-api-postgres-5dbcdc9df5-4qsql   1/1     Running   0          26m
NAME                                CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p          3m           69Mi
notes-api-545877b456-hqjzh          2m           69Mi
notes-api-postgres-5dbcdc9df5-4qsql   8m           19Mi
ubuntu@ip-172-31-33-133:~$ echo "INCIDENT 3 START (T+0): $(date -u '+%Y-%m-%d %H:%M:%S UTC')"
cd ~/SRE-Project1
./tests/simulate-high-cpu.sh
INCIDENT 3 START (T+0): 2026-08-02 23:12:52 UTC
========================================
 FAILURE SIMULATION: High CPU Exhaustion
 Start time: 2026-08-02 23:12:52 UTC
========================================
[STEP 1] Baseline — current pod CPU (via kubectl top):
NAME                        CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p   2m           69Mi
notes-api-545877b456-hqjzh   3m           69Mi

[STEP 2] Generating sustained CPU load via /api/stress...
         Sending 12 rounds of stress (each burns CPU for ~20s).
         Running them in parallel to hit both replicas.

  Round 1: firing parallel stress requests (23:12:52)
  Round 2: firing parallel stress requests (23:13:12)
  Round 3: firing parallel stress requests (23:13:32)
  Round 4: firing parallel stress requests (23:13:52)
  Round 5: firing parallel stress requests (23:14:12)
  Round 6: firing parallel stress requests (23:14:32)
  Round 7: firing parallel stress requests (23:14:52)
  Round 8: firing parallel stress requests (23:15:12)
  Round 9: firing parallel stress requests (23:15:32)

[STEP 3] Load generation complete.
[INFO] Detection: watch Prometheus Alerts for 'NotesApiHighCPU'.
[INFO] Watch the CPU rise in Grafana (Node Exporter / app panels).
[INFO] Recovery is automatic — CPU returns to normal once stress stops.

[STEP 4] Post-load pod CPU:
NAME                        CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p   501m         70Mi
notes-api-545877b456-hqjzh   501m         69Mi

 End time: 2026-08-02 23:15:58 UTC
```

High CPU observed

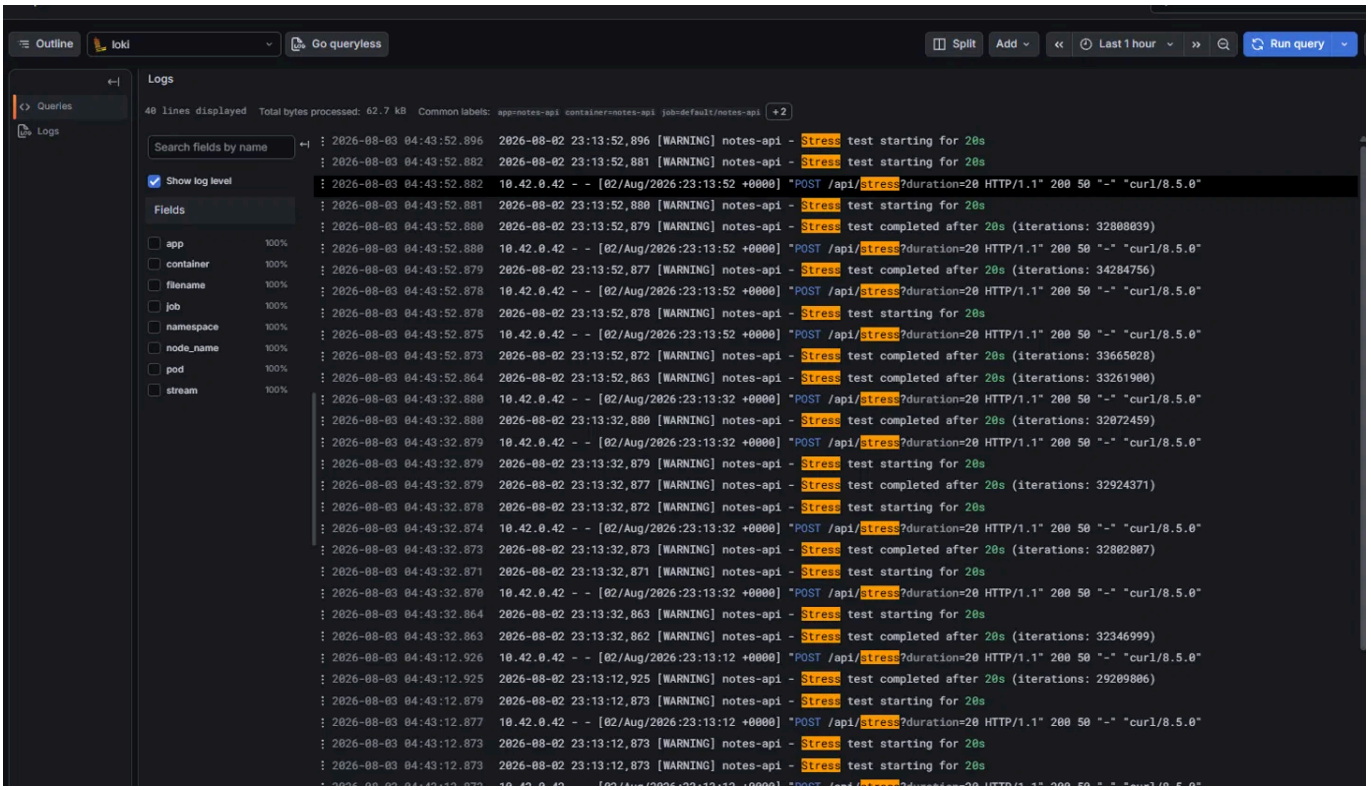
```
Every 2.0s: sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -n default

NAME                                CPU(cores)   MEMORY(bytes)
notes-api-545877b456-7965p          501m         69Mi
notes-api-545877b456-hqjzh          501m         70Mi
notes-api-postgres-5dbcdc9df5-4qsql   9m           19Mi
```

Alert firing

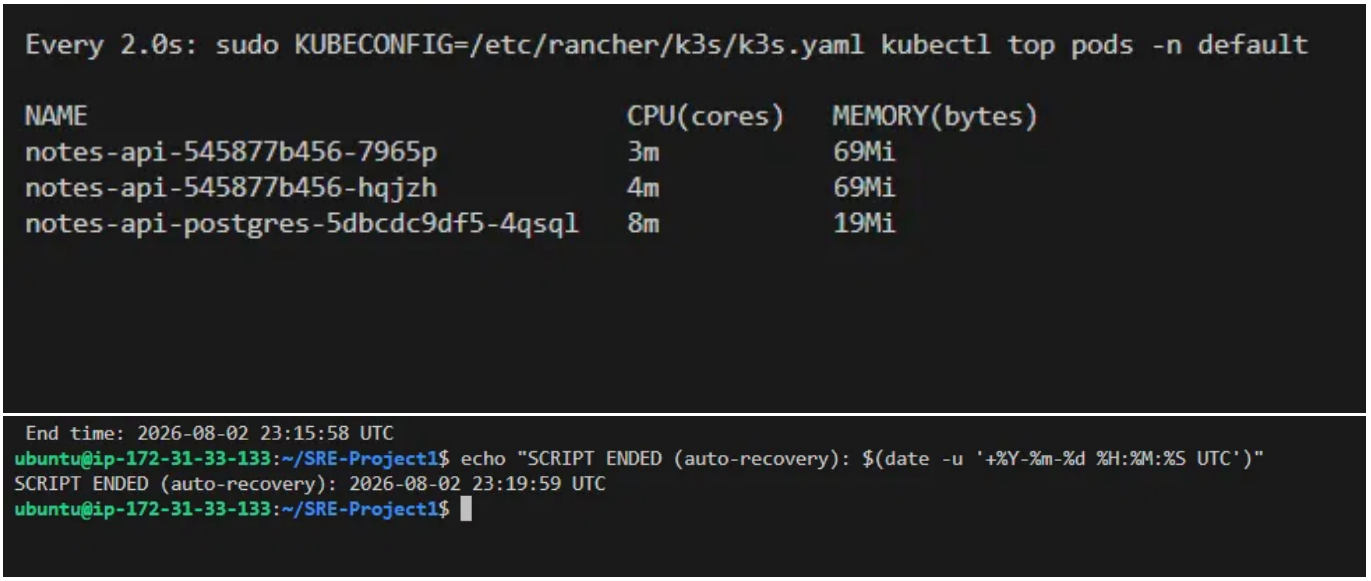




Logs (Loki)



Recovery



10. Reproduction

```
ssh ubuntu@13.126.63.217
cd ~/SRE-Project1
./tests/simulate-high-cpu.sh
# alert to watch (Prometheus): NotesApiHighCPU
# recovery is automatic — CPU drops to baseline ~30s after the script ends;
# the alert auto-resolves ~3 min later as the rate([3m]) window ages out.

# check CPU during the run:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -l app=notes-api
```